

深度学习正则化

学习目的

偏差与方差

数据集划分

偏差与方差的意义

解决方法

正则化 (Regularization)

逻辑回归的L1与L2正则化

正则化项的理解

神经网络中的正则化

正则化为什么能够防止过拟合

Droupout正则化

Inverted droupout

droupout为什么有效总结

其它正则化方法

早停止法

数据增强

学习目的

- 目标
 - 了解偏差与方差的意义
 - 知道L2正则化与L1正则化的数学意义
 - 知道Droupout正则化的方法
 - 了解早停止法、数据增强法的其它正则化方式
- 应用
 - 无

偏差与方差

数据集划分

首选我们对机器学习当中涉及到的数据集划分进行一个简单的复习

- 训练集：用训练集对算法或模型进行训练过程；
- 验证集：利用验证集进行交叉验证，选择出最好的模型；
- 测试集：最后利用测试集对模型进行测试，对学习方法进行评估

在小数据量的时代，如 100、1000、10000 的数据量大小，可以将数据集按照以下比例进行划分：

- 无验证集的情况：70% / 30%
- 有验证集的情况：60% / 20% / 20%

而在如今的大数据时代，拥有的数据集的规模可能是百万级别的，所以验证集和测试集所占的比重会趋向于变得更小。

- 100 万数据量：98% / 1% / 1%
- 超百万数据量：99.5% / 0.25% / 0.25%

以上这些比例可以根据数据集情况选择。

偏差与方差的意义

"偏差-方差分解"(bias-variance decomposition) 是解释学习算法泛化性能的一种重要工具。

泛化误差可分解为偏差，方差与噪声，泛化性能是由学习算法的能力，数据的充分性以及学习任务本身的难度所共同决定的。

- 偏差：度量了学习算法的期望预测与真实结果的偏离程度，即刻画了学习算法本身的拟合能力
- 方差：度量了同样大小的训练集的变动所导致的学习性能的变化，即刻画了数据扰动所造成的影响
- 噪声：表达了在当前任务上任何学习算法所能够达到的期望泛化误差的下界，即刻画了学习问题本身的难度。

那么偏差，方差与我们的数据集划分到底有什么关系呢？

- 1,训练集的错误率较小，而验证集/测试集的错误率较大，说明模型存在较大方差，可能出现了过拟合
- 2,训练集和测试集的错误率都较大，且两者相近，说明模型存在较大偏差，可能出现了欠拟合
- 3,训练集和测试集的错误率都较小，且两者相近，说明方差和偏差都较小，这个模型效果比较好。

所以我们最终总结，方差一般指的是数据模型得出来了，能不能对未知数据的扰动预测准确。而偏差说明在训练集当中就已经误差较大了，基本上在测试集中没有好的效果。

所以如果我们的模型出现了较大的方差或者同时也有较大的偏差，该怎么去解决？

- 过拟合：训练集好，测试集不行，模型参数过于复杂
 - 训练集的数据分布没有代表性
- 欠拟合：训练集不好，测试集也不好，模型、数据都有可能

解决方法

对于高方差，有以下几种方式：

- 获取更多的数据，使得训练能够包含所有可能出现的情况
- 正则化
- 寻找更合适的网络

对于高偏差，有以下几种方式：

- 扩大网络规模，如添加隐藏层或者神经元数量
- 寻找合适的网络架构，使用更大的网络结构，如AlexNet
- 训练时间更长一些

不断尝试，直到找到低偏差、低方差的框架

正则化（Regularization）

正则化，即在成本函数中加入一个正则化项（惩罚项），惩罚模型的复杂度，防止网络过拟合

逻辑回归的L1与L2正则化

逻辑回归的参数W数量根据特征的数量而定，那么正则化如下

- 逻辑回归的损失函数中增加L2正则化

$$J(w, b) = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2m} \|w\|_2^2$$

$$\text{其中的L2范数可以理解: } \frac{\lambda}{2m} \|w\|_2^2 = \frac{\lambda}{2m} \sum_{j=1}^n w_j^2 = \frac{\lambda}{2m} w^T w$$

解释：所有w参数的平方和的结果

- 逻辑回归的损失函数中增加L1正则化

$$J(w, b) = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2m} \|w\|_1$$

其中L2范数可以理解为: $\frac{\lambda}{2m} \|w\|_1 = \frac{\lambda}{2m} \sum_{j=1}^n |w_j|$

其中, λ 为正则化因子, 是**超参数**。

正则化项的理解

在损失函数中增加一项, 那么其实梯度下降是要减少损失函数的大小, 对于L2或者L1来讲都是要去减少这正则项的大小, 那么也就是会减少W权重的大小。这是我们一个直观上的感受。

- 接下来我们通过方向传播来理解这个其中的L2, 对于损失函数我们要反向传播求参数梯度:

$$(1) dW = \frac{\partial L}{\partial w} + \frac{\lambda}{m} W$$

前面的默认损失函数的梯度计算结果默认为backprop, 那么更新的参数就为

$$(2) W := W - \alpha dW \text{ 那么我们将第一个公式带入第二个得到}$$

$$\rightarrow W := W - \alpha \left(\frac{\partial L}{\partial w} + \frac{\lambda}{m} W \right)$$

$$\rightarrow W = W - \frac{\alpha \lambda}{m} W - \alpha * \frac{\partial L}{\partial w}$$

所以每次更新的时候都会让 $W(1 - \frac{\alpha \lambda}{m})$, 这个系数永远小于1, 所以我们通常称L2范数为**权重衰减**。

更新参数: 在原来的基础上, 增加了一个值, 使得w变得更小一些

公式理解: 梯度更新的时候, w多减去了一项值 (惩罚2项进行导数计算得来的)

神经网络中的正则化

神经网络中的正则化与逻辑回归相似, 只不过参数w变多了, 每一层都有若干个权重, 可以理解成一个矩阵

$$J(w^{[1]}, b^{[1]}, \dots, w^{[L]}, b^{[L]}) = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2m} \sum_{l=1}^L \|w^{[l]}\|_F^2$$

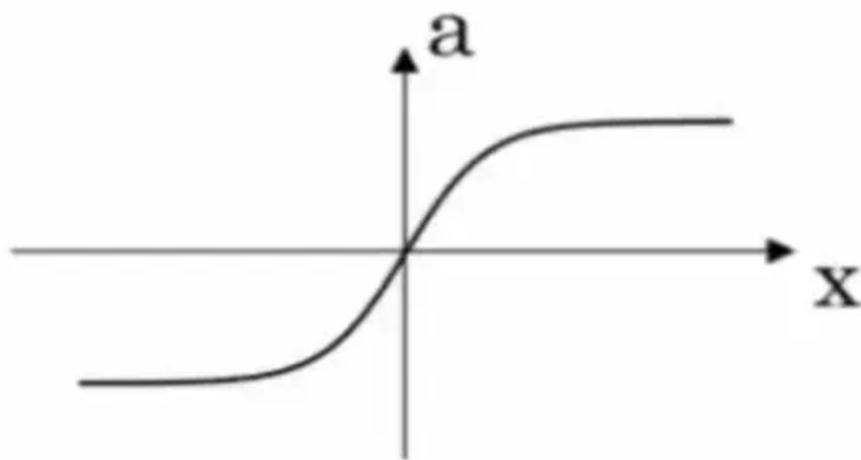
我们把 $w[l]$ 理解某一层神经元的权重参数，其中这是加入了L2范数，可以是

$$\|w[l]\|_F^2 = \sum_{i=1}^{n^{[l-1]}} \sum_{j=1}^{n^{[l]}} (w_{ij}^{[l]})^2$$

对于矩阵的L2范数，有个专业名称叫弗罗贝尼乌斯范数（Frobenius Norm）

正则化为什么能够防止过拟合

正则化因子设置的足够大的情况下，为了使成本函数最小化，权重矩阵 w 就会被设置为接近于0的值。直观上相当于消除了很多神经元的影响，那么大的神经网络就会变成一个较小的网络。



在加入正则化后，当 λ 增大导致 $W[l]$ 减小， $Z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$ ，便会减小。由上图可知，在 z 较小（接近于0）的区域里，函数近似线性，所以每层的函数就近似线性函数，整个网络就成为一个简单的近似线性的网络，因此不会发生过拟合。

Droupout正则化

Droupout论文地址：

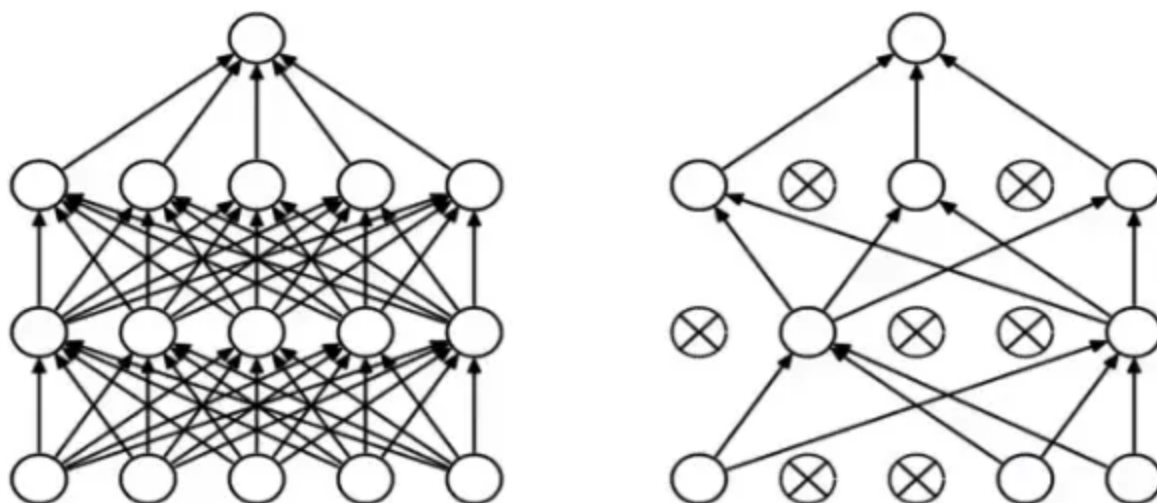
<http://mlr.org/papers/volume15/srivastava14a.old/srivastava14a.pdf>

地址

Abstract

Deep neural nets with a large number of parameters are very powerful machine learning systems. However, overfitting is a serious problem in such networks. Large networks are also slow to use, making it difficult to deal with overfitting by combining the predictions of many different large neural nets at test time. Dropout is a technique for addressing this problem. The key idea is to randomly drop units (along with their connections) from the neural network during training. This prevents units from co-adapting too much. During training, dropout samples from an exponential number of different “thinned” networks. At test time, it is easy to approximate the effect of averaging the predictions of all these thinned networks by simply using a single unthinned network that has smaller weights. This significantly reduces overfitting and gives major improvements over other regularization methods. We show that dropout improves the performance of neural networks on supervised learning tasks in vision, speech recognition, document classification and computational biology, obtaining state-of-the-art results on many benchmark data sets.

Droupout: 随机的对神经网络每一层进行丢弃部分神经元操作



对于网络的每一层会进行设置保留概率，即keep_prob.假设keep_prob为0.8，那么也就是每一层所有神经元有20%的概率直接失效，可以理解为0.

Inverted droupout

这种方式会对每层进行如下代码操作

```
# 假设设置神经元保留概率
keep_prob = 0.8
# 随机建立一个标记1 or 0的矩阵，表示随机失活的单元，占比20%
dl = np.random.rand(al.shape[0], al.shape[1]) < keep_prob
# 让al对应dl的为0地方结果为0
al = np.multiply(al, dl)

# 为了测试的时候，每一个单元都参与进来
al /= keep_prob
```

```
# 假设设置神经元保留概率
keep_prob = 0.8
# 随机建立一个标记1 or 0的矩阵，表示随机失活的单元，占比20%
dl = np.random.rand(al.shape[0], al.shape[1]) < keep_prob
# 让a1对应dl的为0地方结果为0
al = np.multiply(al, dl)

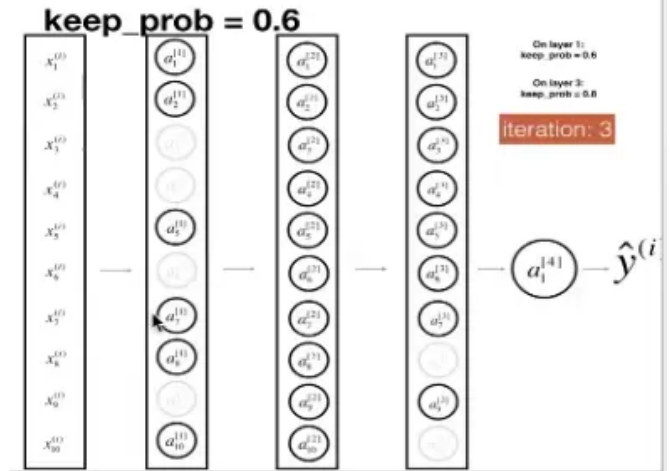
# 为了测试的时候，每一个单元都参与进来
al /= keep_prob
```

第一层10个神经元， 参数： $W = 6 * 10$

[1, 10]

[6个1, 4个0]

6个值保留下来，4个值直接没了



- 训练的时候只有占比为p的隐藏层单元参与训练
- 增加最后一行代码的原因，在预测的时候，所有的隐藏层单元都需要参与进来，就需要测试的时候将输出结果除以p使下一层的输入规模保持不变

假设keep_prob=p=0.8

$z^l = w^l a^{l-1} + b^l$ ，当 $l-1$ 层有比例为 $1-p=0.2$ 单元drop后， a^{l-1} 大约会变为原来的80%，为了保证 l 层的 z 值期望（可以理解为均值）不变，所以要在 a^{l-1} 与dropout矩阵乘积后的权重进行扩大。要乘以 $\frac{1}{p} = 10/8$ (增大)

注：原始：(1+1+1+1+1+1+1+1+1+1)/10 = 1,现在其中20%失效，则平均值为0.8，所以0.8 * (10/8) = 1. 相当于其中8个神经元参数增大了(10/8)倍

测试期间，平均的量没变

dropout为什么有效总结

加入dropout后，输入的特征都存在被随机清除的可能，所以该神经元不会再特别依赖于任何一个输入特征，也就是不会给任何一个输入特征设置太大的权重。通过传播过程，dropout将产生和L2正则化相同的收缩权重的效果。

- 对于不同的层，设置的keep_prob大小也不一致，神经元较少的层，会设keep_prob为1.0，而神经元多的层则会设置比较小的keep_prob

- 通常被使用再计算机视觉领域，图像拥有更多的特征，场景容易过拟合，效果被实验人员证明是不错的。

调试时候使用技巧：

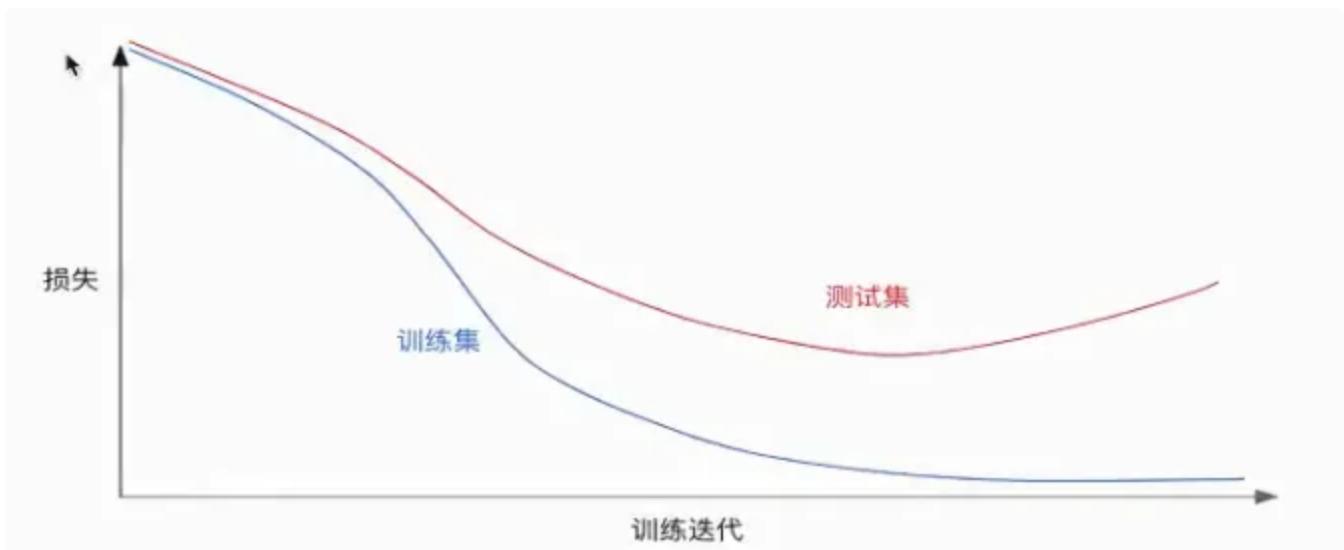
dropout的缺点是成本函数无法被明确定义，因为每次会随机消除一部分神经元，所以参数也无法确定具体哪一些，在反向传播的时候带来计算上的麻烦，也就无法保证当前网络是否损失函数下降的。如果要使用droupout,会先关闭这个参数，保证损失函数是单调下降的，确定网络没有问题，再次打开droupout才会有效。

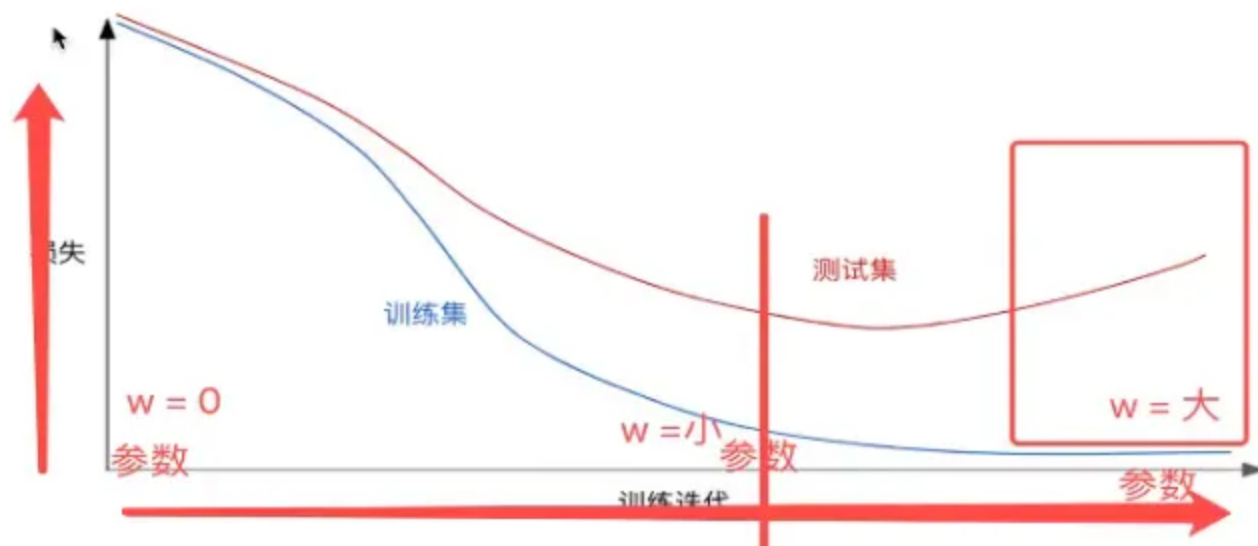
其它正则化方法

- 早停止法
- 数据增强

早停止法

通常我们在训练验证的时候，发现过拟合，可以得到下面这张损失图





通常不断训练之后，损失越来越小。但是到了一定之后，模型学到的过于复杂（过于拟合训练集上的数据的特征）造成测试集开始损失较小，后来又变大。模型的 w 参数会越来越大，那么可以在测试集损失减小一定程度后停止训练。

但是这种方法治标不治本，得从根本上解决数据或者网络的问题。

数据增强

指透过剪切，旋转/反射/翻转变换，缩放变换，平移变换，尺度变换，对比度变换，噪声扰动，颜色变换等一种或多种组合数据增强变换的方式来增加数据集的大小。



即使卷积神经网络被放在不同方向上，卷积神经网络对平移，视角，尺寸或照度（或以上组合）保持不变性，都会认为是一个物体。

- 为什么这样做？

假设数据集中的两个类。左边的代表品牌A（福特），右边的代表品牌B（雪佛兰）。

假设完成了训练，并且输入下面的图像（品牌A），但是你的神经网络输出认为它是品牌B的汽车！



为什么会发生这种现象？因为算法可能会寻找区分一个类和另一个类的最明显特征。在这个例子中，这个特征就是所有品牌A的汽车朝向左边，所有品牌B的汽车朝向右边。神经网络的好坏取决于输入的数据。

怎么解决这个问题？

我们需要减少数据集中不相关特征的数量。对上面的汽车类型分类器来说，你只需要将现有的数据集中的照片水平翻转，使汽车朝向另一侧。现在，用新的数据集训练神经网络，通过增强数据集，可以防止神经网络学习到不相关的模式，提升效果。（在没有采集更多的图片前提下）

- 数据增强类别

那么我们应该在机器学习过程中的什么位置进行数据增强？在向模型输入数据之前增强数据集。

- 离线增强。预先进行所有必要的变换，从根本上增加数据集的规模（例如，通过翻转所有图像，保存后数据集数量会增加2倍）
- 在线增强，或称为动态增强。可通过对即将输入模型的小批量数据的执行相应的变化，这样同一张图片每次训练被随机执行一些变化操作，相当于不同的数据集了。

那么我们的代码中也会进行这种在线增强。

数据增强技术

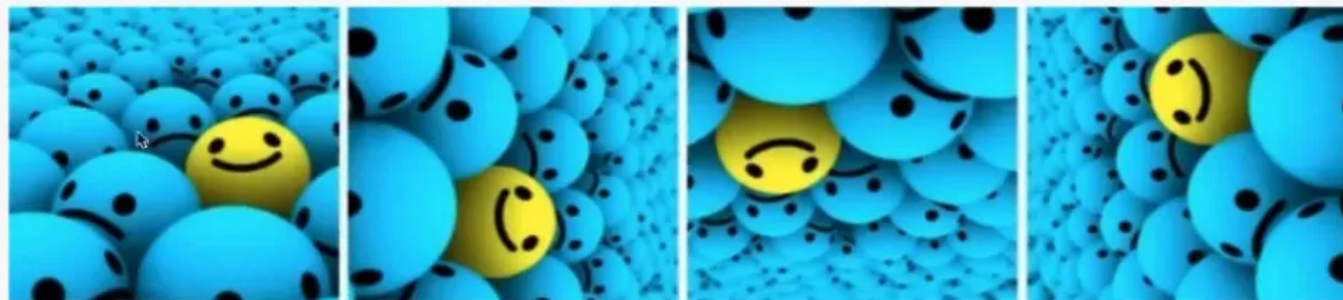
下面一些方法基础但功能强大的增强技术，目前被广泛应用。

翻转：`tf.image.random_flip_left_right`

你可以水平垂直翻转图像。一些架构并不支持垂直翻转图像。但，垂直翻转等价于将图片旋转180再水平翻转。例子：



旋转：rotate



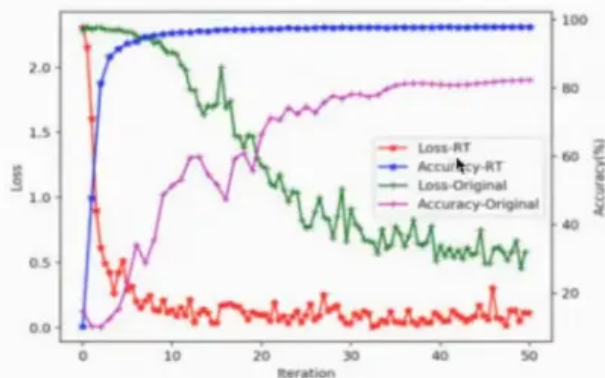
剪裁：random_crop

随机从原始2图像中采样一部分，然后将这部分图像调整为原始图像大小。这个方法更流行的叫法是随机裁剪

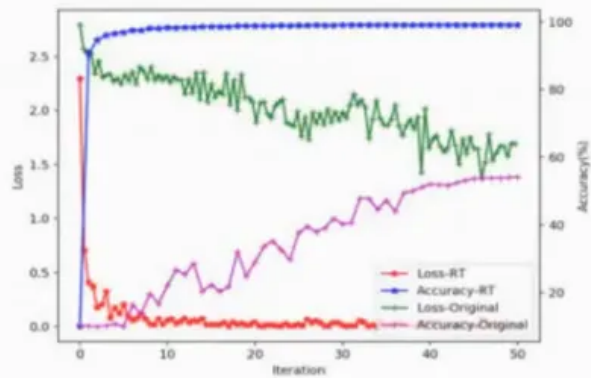


平移、缩放等方法：

数据增强的效果是非第好的，比如下面的例子，绿色和粉色表示没有数据增强之前的损失和准确率效果，红色和蓝色表示数据增强之后的损失和准确率结果，可以看到学习效果也改善较快。



(c) AlexNet-MNIST



(d) GoogLeNet-MNIST

那么tensorflow 官方源码都是基于v99与incaption论文的图像增强介绍，全部通过tt.image相关api来预处理图像。并且提供了各种封装过timage之后的apl。那么tensorflow 官网也给我们提供了一些模型的数据增强过程。

